SOFTWARE ARCHITECTURE

AND

DESIGN SPECIFICATION

SMART COMPLAINT MANAGEMENT SYSTEM

Version: 1.0

Status: Draft

Date: 09 October 2026

Group No.: 8

Course: Software Engineering (UE24CS341A)

Faculty: Pradeep Kumar Kenny, Associate Professor

Department: Computer Science and Engineering

Institution: PES University

Prepared by

1. Keerthana M — PES2UG24CS638
2. Raksha — PES2UG24CS652
3. Dhruva Ganesh Hegde — PES2UG24CS671
4. ShivaSwaroop D S — PES2UG24CS679

Revision History

Version	Date	Author	Change Summary
1.0	09-10-2026	Group 8	Initial Software Architecture and Design Specification for the Smart Complaint Management System.

Approvals

Role	Name	Signature / Date
Team Member	Keerthana M	
Team Member	Raksha	
Team Member	Dhruva Ganesh Hegde	
Team Member	ShivaSwaroop D S	
Faculty / Course Coordinator	Pradeep Kumar Kenny	

1. Introduction

1.1 Purpose

This document describes the software architecture and design of the Smart Complaint Management System (SCMS). It builds on the Software Requirements Specification (SRS) and explains the organization of the system into components, the interactions between users and administrators, the major workflows, and the design decisions followed during development.

The document provides a common reference for all four team members while implementing and integrating their respective modules. It also describes the system's technology choices, security considerations, interfaces, error-handling mechanisms and design models. The objective is to ensure that the application is modular, understandable, maintainable and consistent with the functional and non-functional requirements identified in the SRS.

1.2 Scope

The Smart Complaint Management System is a centralized application designed to simplify the process of submitting, tracking, managing and resolving complaints. It provides a structured workflow through which users can submit complaints and monitor their progress, while authorized administrators can review complaints, update their status, provide resolutions and close completed complaints.

The scope of the system includes:

- User registration, authentication and logout.
- Complaint submission with mandatory-field validation.
- Generation of a unique complaint ID for each successfully submitted complaint.
- Storage and retrieval of complaint information.

- Viewing submitted complaints and tracking their current status.
- Displaying complaint updates, responses and final resolutions.
- Administrator authentication and complaint management.
- Updating complaint status and recording resolutions.
- Closing complaints after resolution.
- Protecting user accounts and complaint information through appropriate security controls.

The system is intended for an academic project demonstration. Integration with external complaint-management platforms and production-scale infrastructure are outside the current scope unless explicitly added to the approved requirements.

1.3 Audience

This document is intended for the following stakeholders:

- **Project Development Team:** To understand the system architecture, component responsibilities, interfaces and implementation decisions.
- **Course Faculty and Evaluators:** To review the architectural design, requirement coverage, security considerations and overall technical approach.
- **Testers:** To understand system workflows, component interactions and expected behaviour while preparing and executing tests.
- **System Administrators:** To understand the administrative functions and the controls required to manage complaints securely.
- **Future Maintainers:** To use the documented architecture and design decisions when correcting defects, modifying existing functionality or extending the system.

1.4 Definitions

- **SCMS:** Smart Complaint Management System; the application used to submit, track, manage and resolve complaints.
- **SRS:** Software Requirements Specification; the document describing the system's functional and non-functional requirements.
- **SAD:** Software Architecture and Design Specification; the document describing the system's architecture, components, interfaces and design decisions.

- **STP:** Software Test Plan; the document defining the testing scope, strategy, resources and responsibilities.
- **RTM:** Requirements Traceability Matrix; a mapping between requirements, system modules and associated test cases.
- **FR:** Functional Requirement; a statement describing a function that the system must provide.
- **NFR:** Non-Functional Requirement; a statement describing a quality attribute or constraint, such as performance, usability, reliability or security.
- **API:** Application Programming Interface; an interface that allows software components to communicate.
- **REST:** Representational State Transfer; an architectural style commonly used for web APIs.
- **CRUD:** Create, Read, Update and Delete; the basic operations used to manage stored information.
- **UML:** Unified Modeling Language; a standard notation used to represent software structure and behaviour.
- **RBAC:** Role-Based Access Control; a mechanism that restricts access according to a user's assigned role.
- **HTTP:** Hypertext Transfer Protocol; a protocol used for communication between web clients and servers.
- **Authentication:** The process of verifying a user's identity.
- **Authorization:** The process of determining which resources and operations an authenticated user is permitted to access.

2. Document Overview

2.1 How to Use This Document

This document is organized into sections describing the architecture and design of the Smart Complaint Management System.

Section 1 introduces the project, its purpose, scope, intended audience and terminology.

Section 2 explains the organization of the document and identifies the related project documents.

Section 3 describes the system architecture, including architectural goals and constraints, stakeholders, the UML component diagram, component responsibilities, the selected architecture pattern, technology stack, risks, requirements traceability and security architecture.

Section 4 describes the detailed system design, including the design overview, UML sequence diagrams for important workflows, API interfaces, error handling, logging, monitoring, user experience considerations and future improvements.

Section 5 contains the glossary, references and tools used during architecture and design.

The document should be read alongside the SRS and Software Test Plan to understand how the proposed architecture supports the specified requirements and how the system's behaviour will be verified.

2.2 Related Documents

The following documents are associated with the Smart Complaint Management System:

1. **Software Requirements Specification (SRS) — Smart Complaint Management System, Version 1.0:** Defines the problem statement, scope, user classes, functional requirements, non-functional requirements, security requirements, acceptance tests and requirements traceability.

2. **Software Test Plan (STP) — Smart Complaint Management System, Version 1.0:** Defines the testing scope, test strategy, test environment, schedule, responsibilities, risks and test deliverables.

3. **Project Submission Guidelines — Software Engineering (UE24CS341A):** Provides the applicable course requirements and submission instructions.

4. **Software Architecture and Design Specification (SAD) — Smart Complaint Management System, Version 1.0:** Documents the architecture, component responsibilities, system interactions, interfaces and design decisions for the project.

3. System Architecture

3.1 Architectural Goals and Constraints

The architecture of the Smart Complaint Management System is designed to provide a reliable, secure and maintainable platform for complaint submission, tracking and resolution. The system separates the user interface, application logic and data storage so that each part can be developed, tested and maintained independently.

The primary architectural goals are:

1. **Modularity:** Divide the system into independent modules for authentication, complaint submission, complaint tracking, administration and notifications.

2. **Maintainability:** Organize the application into clearly defined components to simplify debugging, modification and future enhancement.

3. **Security:** Protect user accounts and complaint records through authentication, authorization, input validation and secure handling of sensitive information.

4. **Data consistency:** Maintain accurate complaint details, status information, timestamps and resolution records.

5. **Usability:** Provide a straightforward interface that enables users to submit complaints and track their progress easily.

6. **Scalability:** Allow individual components to be enhanced as the number of users and complaints increases.

7. **Testability:** Support independent testing of the user interface, backend services, database operations and administrative workflows.

Architectural Constraints

The architecture is subject to the following constraints:

- The project must be developed within the academic schedule and available resources.
- The system must satisfy the functional and non-functional requirements defined in the SRS.
- Access to administrative operations must be restricted to authorized administrators.
- Complaint information must be stored and retrieved consistently.
- The application must be compatible with the selected development environment and technology stack.
- The design should remain simple enough for implementation and demonstration as a semester project.

3.2 System Stakeholders

The Smart Complaint Management System has the following primary stakeholders:

1. Registered User

A registered user can log in, submit complaints, view previously submitted complaints, track complaint status and review updates or resolutions associated with their complaints.

2. Administrator

The administrator is responsible for viewing complaints, reviewing complaint details, updating their status, recording resolutions and closing complaints after the required action has been completed. Administrative access must be protected by authorization controls.

3. Project Development Team

The four team members are responsible for designing, implementing, testing and integrating the modules assigned to them. The team must ensure that the interfaces between modules remain consistent.

4. Course Faculty and Evaluators

Faculty members and evaluators assess the system's functionality, architecture, documentation, testing and compliance with the academic project requirements.

5. Future Maintainers

Future maintainers may update existing features, fix defects, improve security or extend the system with additional complaint categories, notifications or reporting capabilities.

3.3 Component (UML) Diagram

The component diagram represents the major software components of the Smart Complaint Management System and the relationships between them. It shows how the user and administrator interfaces communicate with the backend API, how backend services handle application operations, and how information is stored in the database and file storage.

The main components are:

- **User Web Interface:** Provides registration, login, complaint submission, complaint tracking and complaint-update viewing.
- **Admin Web Interface:** Provides administrator login, complaint review, status updates, resolution management and complaint closure.
- **Backend API Server:** Receives requests from the interfaces, validates input, applies application rules and coordinates communication with other components.
- **Authentication Service:** Handles user registration, login, logout, password protection and role-based access checks.
- **Complaint Management Service:** Handles complaint submission, validation, unique complaint ID generation and complaint record management.
- **Complaint Tracking Service:** Retrieves complaint information and presents current status, progress and available updates to the relevant user.
- **Admin Management Service:** Supports authorized administrative operations, including complaint review, status changes and resolution recording.
- **Notification Service:** Supports status and resolution notifications. Email delivery is included only if the notification functionality is implemented and configured.
- **MongoDB Database:** Stores user records, complaint details, complaint status history and relevant administrative records.

- **File Storage:** Stores complaint attachments when file-upload functionality is included in the approved requirements.

3.3 Component (UML) Diagram

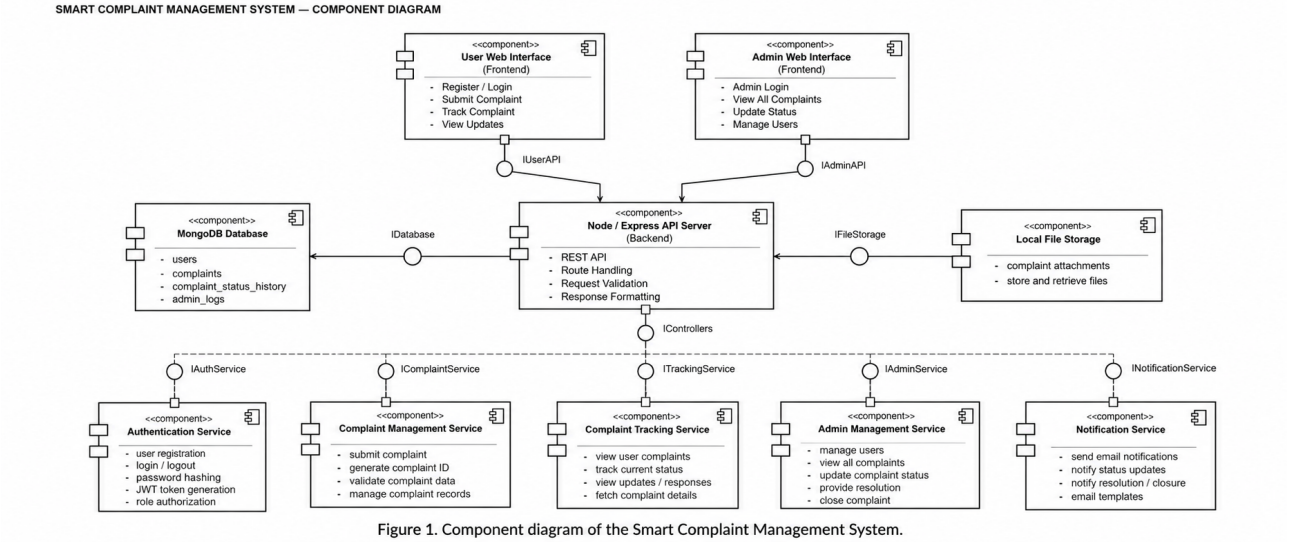


Figure 1. Component diagram of the Smart Complaint Management System.

Sequence Diagrams

The sequence diagrams illustrate the interactions between users, the frontend, backend services, and the database during important system operations. They describe the order in which requests and responses occur during complaint submission and administrative complaint resolution.

The first sequence diagram represents the complaint submission process, including form submission, optional attachment handling, storage of complaint details, and confirmation to the user. The second sequence diagram represents the administrator's workflow for reviewing a complaint, updating its status, recording resolution details, and optionally notifying the concerned user.

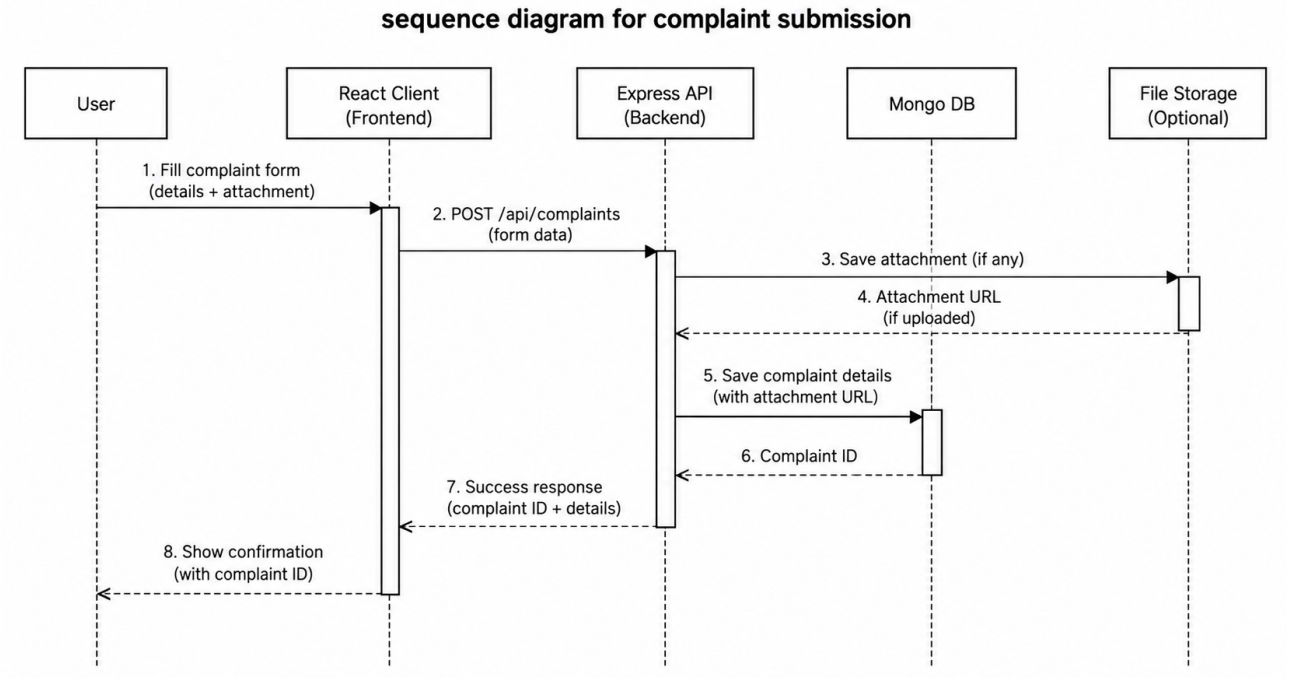


Figure 2. Complaint submission by a user.

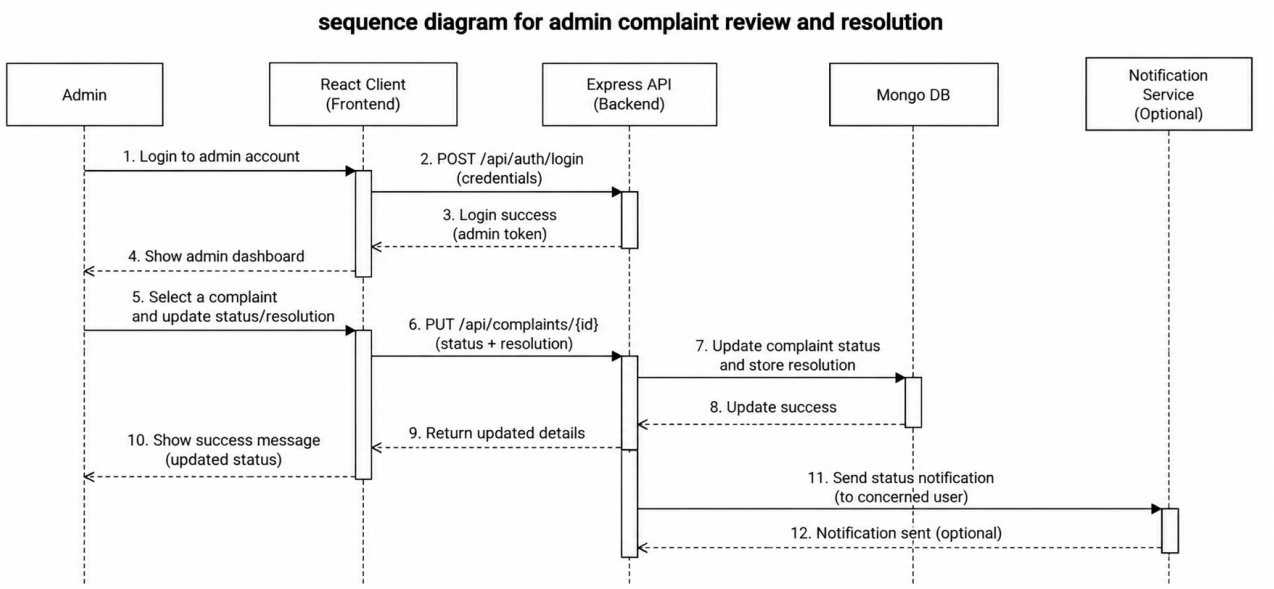


Figure 3. Admin updates complaint status and provides resolution.

3.4 Architectural Pattern

The Smart Complaint Management System follows a layered client-server architecture. The design separates the presentation layer, application layer and data layer. This separation improves modularity, maintainability and the ability to test individual parts of the system.

3.4.1 Presentation Layer

The presentation layer consists of the user and administrator web interfaces. It allows users to interact with the system through forms, buttons, complaint lists and status displays. It collects input and presents responses received from the backend.

3.4.2 Application Layer

The application layer contains the backend API and the services responsible for authentication, complaint management, complaint tracking, administration and notifications. It validates incoming requests, applies business rules and coordinates database operations.

3.4.3 Data Layer

The data layer contains the database and any configured file storage. The database stores user information, complaint records, status history and related data. File storage is used for complaint attachments if this feature is implemented.

3.4.4 Request Processing

When a user submits a complaint, the frontend sends the complaint details to the backend API. The backend validates the request, verifies the user's access rights, generates a complaint identifier and stores the complaint in the database. The system then returns a response indicating whether the submission was successful.

When an administrator updates a complaint, the backend verifies the administrator's permissions, processes the update and stores the new status or resolution details. The updated information becomes available when the relevant user retrieves the complaint.

This architectural pattern separates presentation, processing and persistence responsibilities while keeping communication between components clearly defined.

3.5 Technology Stack

The following technology stack is proposed for the Smart Complaint Management System.

Layer	Technology	Purpose
Frontend	React.js	Builds the interactive user and administrator interfaces.
Backend	Node.js	Provides the server-side execution environment.
Backend Framework	Express.js	Defines API routes and handles HTTP requests.
Database	MongoDB	Stores users, complaints and status information.
API Communication	REST API over HTTP/HTTPS	Enables communication between frontend and backend.
Authentication	Secure password hashing and token-based authentication, if implemented	Protects user accounts and authenticated sessions.
Development Tools	Visual Studio Code, Git and GitHub	Supports coding, version control and collaboration.
Diagramming Tool	draw.io or an equivalent UML tool	Supports the preparation of architecture and design diagrams.

The listed technologies describe the intended implementation stack. Optional features and specific libraries will be finalized according to the team's implementation and available development environment.

3.6 Architectural Risks and Mitigation

The following risks may affect the development and operation of the system.

Risk	Possible Impact	Mitigation
Invalid user input	Incorrect or incomplete complaint records	Apply frontend and backend validation.
Unauthorized administrative access	Unauthorized modification of complaints	Enforce authentication and server-side authorization.

Database connectivity failure	Complaints may not be saved or retrieved	Handle database errors and return appropriate responses.
Inconsistent complaint status	Users may see outdated or conflicting information	Validate status transitions and persist updates consistently.
Integration issues between modules	Features may fail when combined	Define API contracts and test modules together.
Loss of complaint attachments	Supporting evidence may become unavailable	Validate uploads and use reliable storage practices when attachments are supported.
Schedule constraints	Incomplete features or insufficient testing	Prioritize essential requirements and test incrementally.

3.7 Requirements Traceability

The architecture is intended to support the functional and non-functional requirements specified in the Software Requirements Specification.

Requirement Area	Supporting Components	Verification Method
User registration and login	User Interface, Authentication Service, Database	Registration and authentication tests
Complaint submission	User Interface, Complaint Management Service, Database	Complaint submission and validation tests
Unique complaint identification	Complaint Management Service	Verify identifier generation and uniqueness
Complaint tracking	Complaint Tracking Service, Database	Verify status retrieval and complaint ownership
Administrative complaint management	Admin Interface, Admin Management Service	Administrative workflow tests
Status updates and resolutions	Admin Management Service, Complaint Tracking Service	Verify status and resolution persistence
Access control	Authentication Service, Backend API	Unauthorized-access tests
Data consistency and reliability	Backend API, Database	Data validation and failure-handling tests
Usability and maintainability	Presentation and application layers	Usability review and module-level testing

The traceability mapping connects requirement areas to the architectural components that support them. Exact requirement identifiers and acceptance-test references should match the approved SRS and Software Test Plan.

3.8 Security Architecture

Security is an important consideration because the system stores user information and complaint records. The architecture incorporates the following security measures.

1. **Authentication:** Users must authenticate before accessing protected account and complaint-management operations.

2. **Authorization:** Administrative operations must be restricted to authorized administrators. Ordinary users must not be permitted to update other users' complaints or perform administrative actions.

3. **Password Protection:** Passwords must be stored using an appropriate password-hashing algorithm rather than as plain text.

4. **Input Validation:** The backend must validate incoming data to reduce the risk of malformed requests and injection attacks.

5. **Data Protection:** Sensitive information should be transmitted over HTTPS in a deployed environment.

6. **Error Handling:** Error responses should not expose passwords, tokens, database credentials or unnecessary internal implementation details.

7. **Access Control:** Users should be able to retrieve only the complaint records they are authorized to access.

8. **Administrative Accountability:** Important status changes and resolution actions should be recorded where audit logging is implemented.

These measures define the intended security design. Their effectiveness must be verified during implementation and security testing.

4. Architectural Design Decisions

The architecture of the Smart Complaint Management System is designed to provide a simple, secure, and maintainable platform for complaint registration, tracking, and resolution. The design separates the user interface, application logic, and data storage to make the system easier to develop, test, and extend.

4.1 Technology Stack

The proposed system uses the MERN stack for application development.

1. **Frontend – React.js:** Provides the user interface for registration, login, complaint submission, complaint tracking, and administrative operations.

2. **Backend – Node.js and Express.js:** Handles API requests, validates input, applies application logic, manages authentication, and processes complaint-related operations.

3. **Database – MongoDB:** Stores user details, complaint records, complaint statuses, and resolution information.

4. **File Storage – Optional:** Supports storing complaint-related attachments, such as images or supporting documents, if this feature is implemented.

5. **Version Control – Git and GitHub:** Supports source-code management, collaboration, and tracking changes during development.

4.2 Architectural Rationale

A client-server architecture is selected because it separates the presentation layer from the backend services and database. React provides a responsive interface, while Express and Node.js handle requests and business logic. MongoDB provides flexible document-based storage for complaint records.

This architecture supports modular development, easier maintenance, and future enhancements. The separation of user and administrator operations also helps enforce role-based access control.

4.3 Design Considerations

The following considerations guide the proposed design:

- **Modularity:** Complaint submission, tracking, user account management, and administration are treated as separate functional areas.
- **Security:** Authentication, authorization, input validation, and secure handling of sensitive information are considered during implementation.
- **Reliability:** Complaint records and status changes should be stored consistently, and errors should be handled appropriately.
- **Usability:** Users should be able to submit complaints and understand their current status without unnecessary complexity.
- **Maintainability:** The frontend, backend, and database operations should remain sufficiently separated to simplify testing and future modifications.
- **Scalability:** The design should allow additional complaint categories, reporting features, and notification mechanisms to be introduced later.

5. Conclusion

The Software Architecture and Design specification describes the proposed architecture of the Smart Complaint Management System. It identifies the major components, their responsibilities, their interactions, and the security considerations associated with complaint management.

The proposed client-server architecture separates the frontend, backend, and database responsibilities. The component and sequence diagrams illustrate how complaints are submitted, stored, reviewed, and resolved. The design also considers authentication, authorization, data protection, maintainability, and usability.

This specification provides a foundation for implementation and testing. The final system should be validated against the approved Software Requirements Specification and Software Test Plan to ensure that the implemented functionality satisfies the project requirements.